Solving Chess Endgames

A Reinforcement Learning Approach

Valeria De Stasio, Christian Faccio, Giovanni Lucarelli

September 5, 2025

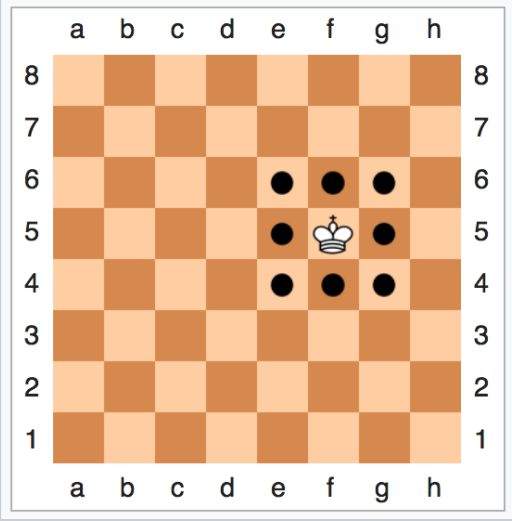
Project Overview

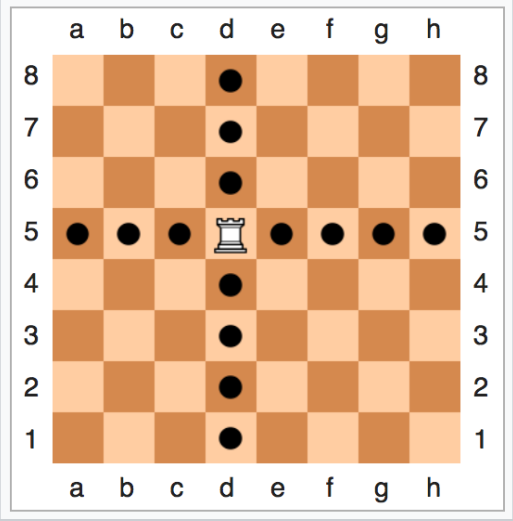
Objectives: Fastest checkmate against optimal playerExperiment with training against random playerComparison of different RL approaches

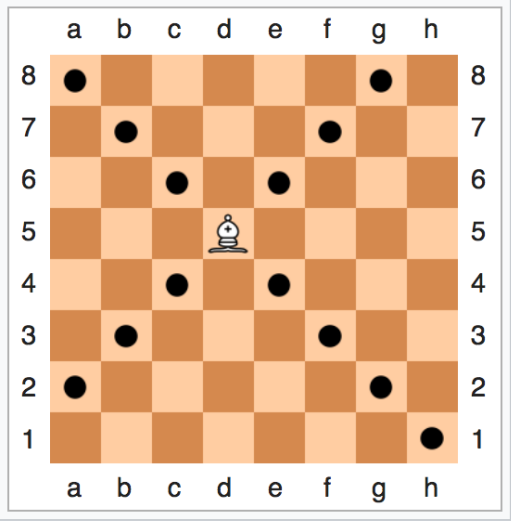
Chess Programming Background

- 1890 El Ajedrecista: electro-mechanical KRK solver
- 1950 Claude Shannon: first article about chess programming
- 1997 *Deep Blue*: first computer program to beat the current world champion
- 2008 *Stockfish*: strong heuristic-based chess engine
- 2017 *Alphazero*: neural network-based approach that defeated Stockfish
- from 2018: more models developed based on *Alphazero*

Chess Rules







- **Win**: checkmate the opponent's king
- **Draw**: stalemate or insufficient material[1]

[1] threefold repetition and the fifty-move rule are not considered in our implementation

Elementary Endgames

- subset of endgames with few pieces left
- foundation for learning more complex endgames
- clear theoretical results: forced checkmate or draw
- Sygyzy endgame tablebases
- examples: KRK Remark: white is always winning in elementary endgames!

MDP Formulation

Markov Game vs MDP

- Chess is a 2-player, turn-based, zero-sum game, formally a Markov Game
- Assume fixed black defender policy: **oracle** or **random**
- Black becomes part of the environment: Markov Game $\rightarrow$ MDP

MDP: States

- State space $\mathcal{S}$: legal pieces positions and side-to-move

```
8/8/K7/8/8/5k2/7R/8 w - - 0 1
```

```
std::array<std::array<U64, 6>, 2> pieces;
Color side_to_move;
```

- Terminal state space $\bar{\mathcal{S}} \subset \mathcal{S}$: checkmate or draw positions (and stm). Defined procedurally

```
state.is_game_over() -> bool
```

MDP: Actions

- Action space $\mathcal{A}(s)$: set of legal moves for the current player.
- Defined procedurally:

```
state.legal_moves() -> std::vector<Move>
```

MDP: Transition

Transition probability $\mathbf{Pr}(S_{t+1} = s' | S_t = s, A_t = a)$

- **Oracle Defender**: deterministic evolution

```
state.do_move(Move)                 // white deterministic
state.do_move(oracle.reply())       // black deterministic
```

- **Random Defender**: stochastic evolution

```
state.do_move(Move)                   // white deterministic
state.do_move(random_reply())         // black random
```

MDP: Reward

Goal: find the fastest mate

Reward:

- checkmate: +1
- draw: -1 (always winning for white)
- encourage faster mates
  - step penalty: -2/-0.01
  - discount factor $\gamma = 0.99$

RL Algorithms

Value Iteration

- $V(s) \leftarrow \max_a \sum_{s',r} p(s', r'|s, a)[r(s, a, s') + \gamma V(s')]$
- $\pi(s) = \arg\max_a \sum_{s',r} p(s'|s, a)[r(s, a, s') + \gamma V(s')]$
- KRk: 182.676 states

Choice of parameters

- Step penalty: -2 $\rightarrow$ values = DTZ
- Draw penalty: -100 $\rightarrow$ slow mates > quick draws
- $\gamma = 1$
- $\theta = 0.0001$

```
'8/8/8/8/8/8/R7/K1k5 w - - 0 1': -15.0,
'8/8/8/8/8/8/1R6/K1k5 w - - 0 1': -15.0,
'8/8/8/8/8/8/3R4/K1k5 w - - 0 1': -15.0,
'8/8/8/8/8/8/4R3/K1k5 w - - 0 1': -15.0,
'8/8/8/8/8/8/5R2/K1k5 w - - 0 1': -15.0,
'8/8/8/8/8/8/6R1/K1k5 w - - 0 1': -15.0,
'8/8/8/8/8/8/7R/K1k5 w - - 0 1': -15.0,
'8/8/8/8/8/R7/8/K1k5 w - - 0 1': -15.0,
'8/8/8/8/8/1R6/8/K1k5 w - - 0 1': -15.0,
'8/8/8/8/8/3R4/8/K1k5 w - - 0 1': -21.0,
'8/8/8/8/8/4R3/8/K1k5 w - - 0 1': -19.0,
'8/8/8/8/8/5R2/8/K1k5 w - - 0 1': -17.0,
```

Random Player

Idea: train against an opponent that plays at random and see how well it can play against an optimal one. Put somehow justification for Qlearning/random player

Q-Learning: model-free approach

- $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$
- $\pi(s) = \arg\max_a Q(s,a)$
- Assume no information on transitions!
- Off-policy, online and one-step.

Choice of parameters

- Max steps per episode: 50 $\rightarrow$ if it's too far from mate, agent may never reach terminal state
- Step penalty: -0.01
- Draw penalty: -1
- $\gamma = 0.99$
- Learning rate $\alpha = 0.05 \rightarrow$ smaller variance
- ϵ-greedy policy with $\epsilon = 0.3$ with decay

Approximate solution methods

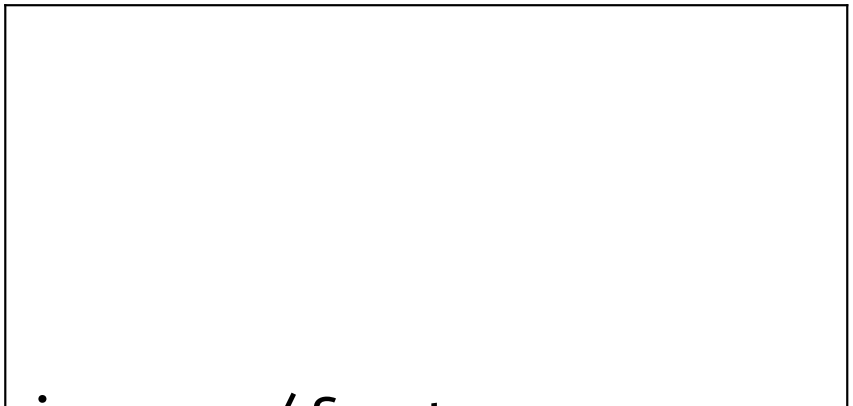
- Non-elementary chess endgames $\rightarrow$ **curse of dimensionality**
- Use one-step actor-critic
- Update rules:
  - $\delta = r + \gamma V(s'; w) - V(s; w)$
  - $\theta \leftarrow \theta + \alpha^{\theta} \delta \nabla$

theta $\log \pi(a|s; \theta)$

- $w \leftarrow w + \alpha^{w} \delta \nabla_{w} V(s; w)$

Approximations

- Policy function approximation: neural network
- State space approximation: linear combination of features
- Feature engineering:
  - Manhattan distance between kings
  - Manhattan distance between black king and white rook
  - File and rank of the kings
  - Distance of the Black King from the side of the board
  - Binary indicator of opposition, parity



Monte Carlo Tree Search

- One of the first methods to solve chess
- No need to train!
- Heuristic methods

Results

Assessment Metrics Overview

* DTM, DTZ etc * success, top1 etc

Why no convergence?

- Difficulties:
  - too many possible actions at each step
  - ideal opponent that never makes mistakes
  - very sparse rewards in an episode
- Possible solutions:
  - more training time
  - implement draw by repetition and 50-move rule $\rightarrow$ more frequent terminal states

Policy interpretability through human chess principles

Video Animation of optimal ply vs our policy

maybe for a mate in 5 or more

Future Works

* zobrist hashing and invariance properties in chess endgames to reduce state space cardinality

Why is it so difficult to solve chess endgames?

> *A player can sometimes afford the luxury of an inaccurate move, or even a definite error, in the opening or middlegame without necessarily obtaining a lost position. In the endgame ... an error can be decisive, and we are rarely presented with a second chance.*

- Paul Keres

Thank You!

Why $\gamma = 0.99$?

The longest rkr endgame requires 16 white moves, hence $\gamma^{16} \approx 0.85$ is a good reward for reaching the goal and producing fast mates.